BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN – ĐIỆN TỬ

BÁO CÁO CUỐI KỲ MÔN HỌC
HỆ THỐNG NHÚNG
BOOTLOADER CƠ BẢN QUA UART

MÃ MÔN HỌC: EMSY435664

SÁNG THỨ 6, ĐỢT 2, HỌC KÌ 1, NĂM HỌC 2025-2026

Nhóm thực hiện: NHÓM 13

Nguyễn Trọng Nhân	**23119182**
Phạm Trọng An Nam	**23119174**
Phan Ngọc Tuấn Nguyên	**23119178**
Hà Quang Minh	**23119169**

Giảng viên hướng dẫn: ThS. Huỳnh Hoàng Hà

Thành phố Hồ Chí Minh, Tháng 1 năm 2026

BẢNG GIẢI TRÌNH CHỈNH SỬA BÁO CÁO
ĐỒ ÁN MÔN HỌC

1. Tên đề tài: Bootloader cơ bản thông qua UART.

2. Nhóm sinh viên thực hiện: Nhóm 13 – Sáng thứ 6, đợt 2, HK1

Nguyễn Trọng Nhân	MSSV: 23119182
Phạm Trọng An Nam	MSSV: 23119174
Phan Ngọc Tuấn Nguyên	MSSV: 23119178
Hà Quang Minh	MSSV: 23119169

3. Tổng hợp các yêu cầu chỉnh sửa báo cáo của giảng viên:

- Giải thích địa chỉ bắt đầu của chương trình Bootloader và Application.
- Giải thích cơ chế xóa dữ liệu bằng cách chuyển cả trang từ bit ‘0’ lên bit ‘1’.
- Cho ví dụ về gói tin dữ liệu .bin, cách thức chuyển đổi gói tin .hex sang .bin.
- Giải thích cách thức hoạt động của các hàm đã sử dụng trong lưu đồ.

4. Giải trình chỉnh sửa báo cáo:

TT	Nội dung góp ý của GV	Kết quả chỉnh sửa bổ sung
1	Giải thích địa chỉ bắt đầu của chương trình Bootloader và Application	Đã bổ sung tại trang 1-2 và trang 24-25
2	Giải thích cơ chế xóa dữ liệu bằng cách chuyển cả trang từ bit ‘0’ lên bit ‘1’	Đã bổ sung tại trang 2
3	Cho ví dụ về gói tin dữ liệu .bin, cách thức chuyển đổi gói tin .hex sang .bin	Đã bổ sung tại trang 6-7
4	Giải thích cách thức hoạt động của các hàm đã sử dụng trong lưu đồ	Đã bổ sung ở phần 2.2 và 3.3

MỤC LỤC

MỤC LỤC ..... **ii**

DANH MỤC HÌNH ẢNH ..... **iv**

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT **1**

1.1. Tổng quan về bootloader 1

1.2. Kiến trúc bộ nhớ Flash của STM32 1

1.2.1. Cấu trúc tổng quan bộ nhớ Flash *1*

1.2.2. Cơ chế đọc - xóa bộ nhớ flash trên STM32F103C8T6 ..... *2*

1.3. Chế độ Boot và Bảng Vector (Vector Table) 3

1.4. Chuẩn giao tiếp và định dạng tệp tin 4

1.4.1. Chuẩn giao tiếp UART *4*

1.4.2. Định dạng tệp tin *5*

CHƯƠNG 2. THIẾT KẾ CHƯƠNG TRÌNH BOOTLOADER **8**

2.1. Thuật toán chương trình Bootloader 8

2.2. Giải thích chương trình con 10

2.2.1. Hàm ghi Flash *10*

2.2.2. Hàm xóa Flash *11*

2.2.3. Hàm xóa Firmware *12*

2.2.4. Hàm nhận và ghi Firmware *12*

2.2.5. Hàm Is_App_Valid() *14*

2.2.6. Hàm Enter_To_App() *14*

CHƯƠNG 3. THIẾT KẾ CÔNG CỤ PYTHON CẬP NHẬT FIRMWARE QUA UART **16**

3.1. Sơ đồ khối và cấu trúc gói tin 16

3.2. Lưu đồ thuật toán của tool nạp Python 17

3.3. Giải thích hàm trong chương trình 18

3.3.1. Bắt tay và thiết lặp kết nối với MCU. *18*

3.3.2. Đọc dữ liệu firmware và gửi header *19*

3.3.3. Tuyền Header theo từng chunk *20*

CHƯƠNG 4. KẾT QUẢ VÀ ĐÁNH GIÁ 21
4.1. Quy trình cập nhật firmware qua UART sử dụng bootloader 21
4.2. Cập nhật firmware bằng chương trình Python.....23
CHƯƠNG 5. KẾT LUẬN 28
DANH MỤC TÀI LIỆU THAM KHẢO 29

DANH MỤC HÌNH ẢNH

Hình 1-1 Kiến trúc bộ nhớ Flash 1
Hình 1-2 Chế độ boot trên STM32F103C8T6 3
Hình 1-3 Mảng con trỏ Vector table 4
Hình 1-4 Sơ đồ hoạt động của truyền thông UART 4
Hình 1-5 Cấu trúc mỗi dòng trong file .hex 5
Hình 1-6 Ảnh minh họa một file .bin 6
Hình 1-7 Câu lệnh chuyển đổi file .hex thành .bin 7
Hình 2-1 Sơ đồ thuật toán Bootloader 8
Hình 3-1 Sơ đồ khối kết nối tool nạp firmware qua UART 16
Hình 3-2 Cấu trúc gói tin truyền Firmware 16
Hình 3-3 Lưu đồ thuật toán của công cụ nạp firmware lên MCU qua UART 17
Hình 4-1 Các thông số trên phần mềm Flash Loader Demonstrator 21
Hình 4-2 Nạp file .hex vào MCU 22
Hình 4-3 Nạp Booloader thành công 23
Hình 4-4 Chương trình BLINK_LED cơ bản 24
Hình 4-5 Bảng Options for Targets của Application 25
Hình 4-6 Hộp thoại chọn file .bin 26
Hình 4-7 Hộp thoại nhập cổng COM tương ứng với STM32 26
Hình 4-8 Chờ tín hiệu BL_READY từ MCU 27
Hình 4-9 Kết quả nạp firmware 27

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về bootloader

Bootloader là một phần mềm dung lượng nhỏ được lưu trong bộ nhớ Flash của vi điều khiển (MCU). Chức năng của Bootloader là quản lý firmware và quyết định chuyển sang chạy chương trình ứng dụng phù hợp.

Cách Bootloader hoạt động diễn ra như sau:

+ Vi điều khiển được cấp nguồn hoặc reset → Bootloader được thực thi đầu tiên.

+ Bootloader kiểm tra xem hệ thống có yêu cầu cập nhật firmware hay không.

+ *Nếu có*, Bootloader nhận firmware mới → ghi đè trong bộ nhớ flash.

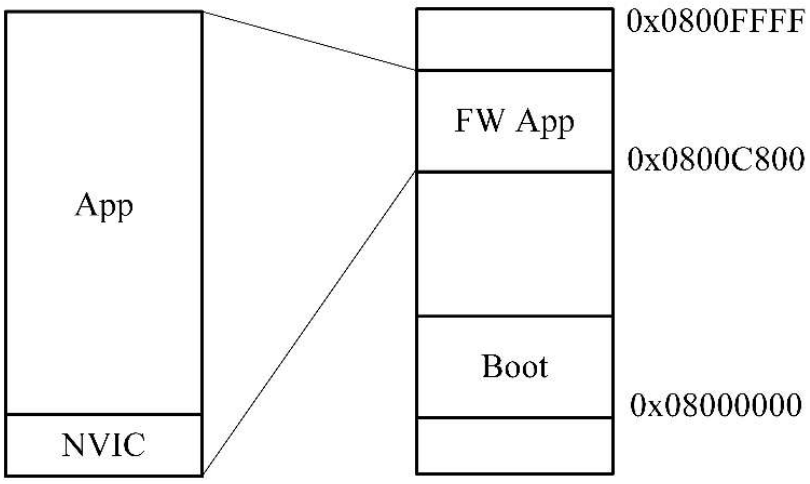
+ *Nếu không có*, Bootloader sẽ chạy firmware ứng dụng cũ có sẵn trên bộ nhớ flash.

1.2. Kiến trúc bộ nhớ Flash của STM32

1.2.1. Cấu trúc tổng quan bộ nhớ Flash

a. Kiến trúc tổng quan

Vi điều khiển STM32F103C8T6 có dung lượng bộ nhớ flash 64 KB, sử dụng cấu trúc single-bank và được tổ chức theo đơn vị Page (mỗi trang 1KB).



Hình 1-1 Kiến trúc bộ nhớ Flash

- Bộ nhớ Flash của MCU được chia thành 2 vùng chính:

+Vùng Boot (Bootloader):

- Địa chỉ bắt đầu: Luôn bắt đầu tại địa chỉ 0x0800 0000 → Địa chỉ cố định của kiến trúc ARM Cortex-M → quy định bởi STMicroelectronics.

- Sau khi khởi động, Bootloader là nơi MCU đọc vector table → thực thi lệnh đầu tiên.
- Địa chỉ kết thúc: Không cố định, phụ thuộc vào dung lượng Bootloader.

+ Vùng FW App (Application / Firmware chính):

- Vùng FW App (Firmware chính) nằm ngay sau vùng Bootloader.
- Địa chỉ bắt đầu nằm tại địa chỉ đầu một trang, không cố định và phụ thuộc vào kích thước Bootloader.
- Địa chỉ kết thúc là cuối Flash, với STM32F103C8T6 là 0x0800 FFFF.
- Dung lượng FW tối đa = dung lượng Flash − kích thước Bootloader.

→ MCU luôn khởi động từ địa chỉ thấp nhất của Flash, sau đó Bootloader sẽ quyết định có chuyển sang chạy Application hay không.

b. Vai trò của NVIC và bảng vector ngắt

NVIC quản lý ngắt và sử dụng vector table (MSP, Reset Handler, ISR) để chuyển luồng thực thi của MCU. Khi Bootloader nhảy sang App:

+ Phải cập nhật lại con trỏ bảng vector ngắt (VTOR) trỏ tới địa chỉ của FW App.

+ Nếu không, ngắt sẽ vẫn trỏ về Boot → gây lỗi hệ thống.

1.2.2. Cơ chế đọc - xóa bộ nhớ flash trên STM32F103C8T6

a. Cơ chế xóa bộ nhớ Flash

- Flash của STM32F103C8T6 được cấu tạo từ floating-gate transistor.

+ Khi ghi: MCU bơm điện tích vào floating gate, làm bit chuyển từ 1 → 0.

+ Khi xóa: MCU rút toàn bộ điện tích khỏi floating gate, các bit trở về 1 (0xFF).

- Để xóa được điện tích, tất cả cell trong page phải xả điện áp xuống mức thấp, nên bắt buộc phải xóa toàn bộ page.

b. Cơ chế ghi bộ nhớ Flash

Bộ nhớ flash lưu giá trị theo kiểu Litte-Endian, mỗi lần ghi được Half-Word (16 bit).

Ví dụ, giá trị 0x1234 khi lưu vào bộ nhớ:

+ Tại địa chỉ N lưu byte thấp 0x34.

+ Tại địa chỉ N+1 lưu byte cao 0x12.

1.3. Chế độ Boot và Bảng Vector (Vector Table)

Chế độ Boot là cơ chế xác định vùng bộ nhớ mà MCU sẽ khởi động và thực thi mã lệnh đầu tiên sau khi reset.

Boot mode selection pins		Boot mode	Aliasing
BOOT1	BOOT0		
x	0	Main Flash memory	Main Flash memory is selected as boot space
0	1	System memory	System memory is selected as boot space
1	1	Embedded SRAM	Embedded SRAM is selected as boot space

Hình 1-2 Chế độ boot trên STM32F103C8T6

+ BOOT0=0: Khởi động từ Flash (0x08000000)→ chạy Bootloader của người dùng.

+ BOOT0 = 1, BOOT1 = 0: Khởi động từ System Memory (ROM).

+ BOOT0 = 1, BOOT1 = 1: Khởi động từ SRAM → để debug/kiểm tra.

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000_0000
-	-3	fixed	Reset	Reset	0x0000_0004
-	-2	fixed	NMI	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000_0008
-	-1	fixed	HardFault	All class of fault	0x0000_000C
-	0	settable	MemManage	Memory management	0x0000_0010
-	1	settable	BusFault	Pre-fetch fault, memory access fault	0x0000_0014
-	2	settable	UsageFault	Undefined instruction or illegal state	0x0000_0018
-	-	-	-	Reserved	0x0000_001C - 0x0000_002B
-	3	settable	SVCall	System service call via SWI instruction	0x0000_002C
-	4	settable	Debug Monitor	Debug Monitor	0x0000_0030
-	-	-	-	Reserved	0x0000_0034
-	5	settable	PendSV	Pendable request for system service	0x0000_0038
-	6	settable	SysTick	System tick timer	0x0000_003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000_0040
1	8	settable	PVD	PVD through EXTI Line detection interrupt	0x0000_0044

Hình 1-3 Mảng con trỏ Vector table

Vector Table là một mảng con trỏ, bắt đầu tại địa chỉ 0x0000 0000. Trong đó, 4 byte đầu tiên chứa giá trị MSP; 4 byte tiếp theo chứa địa chỉ Reset Handler. Ngoài ra, các con trỏ tiếp theo chứa địa chỉ các trình phục vụ ngắt và lỗi.

Vai trò của con trỏ ngăn xếp (MSP) và địa chỉ Reset Handler trong việc chạy chương trình ứng dụng như sau:

- MSP (Main Stack Pointer) là con trỏ chứa địa chỉ bắt đầu của vùng nhớ stack, đảm bảo MCU có vùng stack hợp lệ để gọi hàm và xử lý ngắt trong quá trình thực thi chương trình.

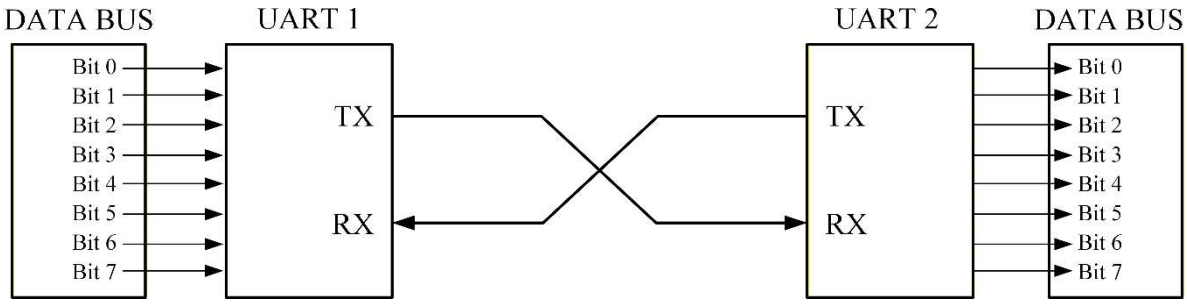
- Reset Handler là địa chỉ bắt đầu thực thi chương trình sau khi reset. MCU nạp địa chỉ này vào thanh ghi PC và thực thi mã khởi động, bao gồm khởi tạo hệ thống và chuyển điều khiển đến hàm main().

1.4. Chuẩn giao tiếp và định dạng tệp tin

1.4.1. Chuẩn giao tiếp UART

a. Định nghĩa

UART (Universal Asynchronous Receiver/Transmitter) là một giao thức truyền thông nối tiếp bất đồng bộ, trong đó dữ liệu được truyền từng bit theo thời gian mà không cần tín hiệu xung clock chung, việc đồng bộ được thực hiện thông qua bit bắt đầu và tốc độ baud đã được cấu hình trước giữa bộ phát và bộ thu.



Hình 1-4 Sơ đồ hoạt động của truyền thông UART

b. Ưu điểm và nhược điểm của giao tiếp UART

Ưu điểm:

- Chỉ cần 2 dây truyền dữ liệu (TX, RX) và chung GND.
- Không cần tín hiệu clock (giao tiếp bất đồng bộ).

- Phần cứng đơn giản, dễ triển khai.
- Có bit chẵn lẻ (parity) để phát hiện lỗi.
- Cấu trúc khung dữ liệu linh hoạt (data bit, parity, stop bit).
- Được hỗ trợ rộng rãi trên hầu hết vi điều khiển.
- Nhiều tài liệu và công cụ hỗ trợ.

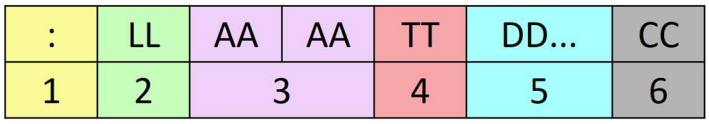
Nhược điểm:

- Kích thước dữ liệu mỗi khung nhỏ (tối đa 9 bit).
- Không hỗ trợ giao tiếp đa thiết bị trên cùng bus (chỉ point-to-point).
- Không có cơ chế xác nhận dữ liệu (ACK/NACK).
- Tốc độ truyền bị giới hạn.
- Yêu cầu baud rate hai bên phải gần nhau (sai lệch nhỏ).

1.4.2. Định dạng tệp tin

a. Tệp tin Intel Hex (.hex)

Mỗi dòng trong file HEX gọi là một bản ghi (record), có dạng: ":LL AAAA TT [DD...] CC"



Hình 1-5 Cấu trúc mỗi dòng trong file .hex

Trong đó: ":"→ Ký tự bắt đầu dòng

"LL" → Số byte dữ liệu trong dòng (hex)

"AAAA" → Địa chỉ bắt đầu của dữ liệu (16-bit)

"TT" → Kiểu dữ liệu (00: Data record, 01: End of file, 02: Extended Segment Address; 04: Extended Linear Address)

"DD..:. → Dữ liệu chương trình (mỗi byte = 2 ký tự hex)

"CC" → Checksum để kiểm tra lỗi dòng dữ liệu

Ví dụ: :10010000214601360121470136007EFE09D2190140

+ : → Kí tự bắt đầu

+ 10 → 16 byte dữ liệu

+ 0100 → Địa chỉ bắt đầu dữ liệu là 0x0100

+ 00 → Type: Data record

+ 21460136012147013600 7EFE09D21901 → Data

+ 40 → Checksum

b. Tệp tin nhị phân (.bin)

Tệp Binary (.bin) chứa dữ liệu nhị phân thô của chương trình.

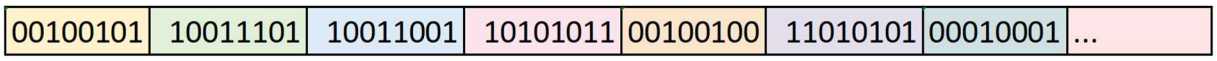
Trong file .bin, không có cấu trúc bản ghi như Intel HEX:

- Không chứa địa chỉ,
- Không có loại dữ liệu,
- Không có checksum.

File .bin chỉ gồm các byte liên tiếp giá trị nhị phân liên tiếp 8 bit.

Khi nạp vào MCU, địa chỉ ghi Flash không nằm trong file mà do công cụ nạp hoặc bootloader chỉ định, các byte trong file sẽ được ghi tuần tự vào các địa chỉ tăng dần.

→ Bootloader đọc/ghi dữ liệu bằng file .bin sẽ không cần biên dịch (bỏ các kí tự bắt đầu, địa chỉ, checksum trong file .hex). Tuy nhiên, cần phải có cơ chế kiểm tra lỗi riêng tránh mất dữ liệu trong lúc truyền.



Hình 1-6 Ảnh minh họa một file .bin

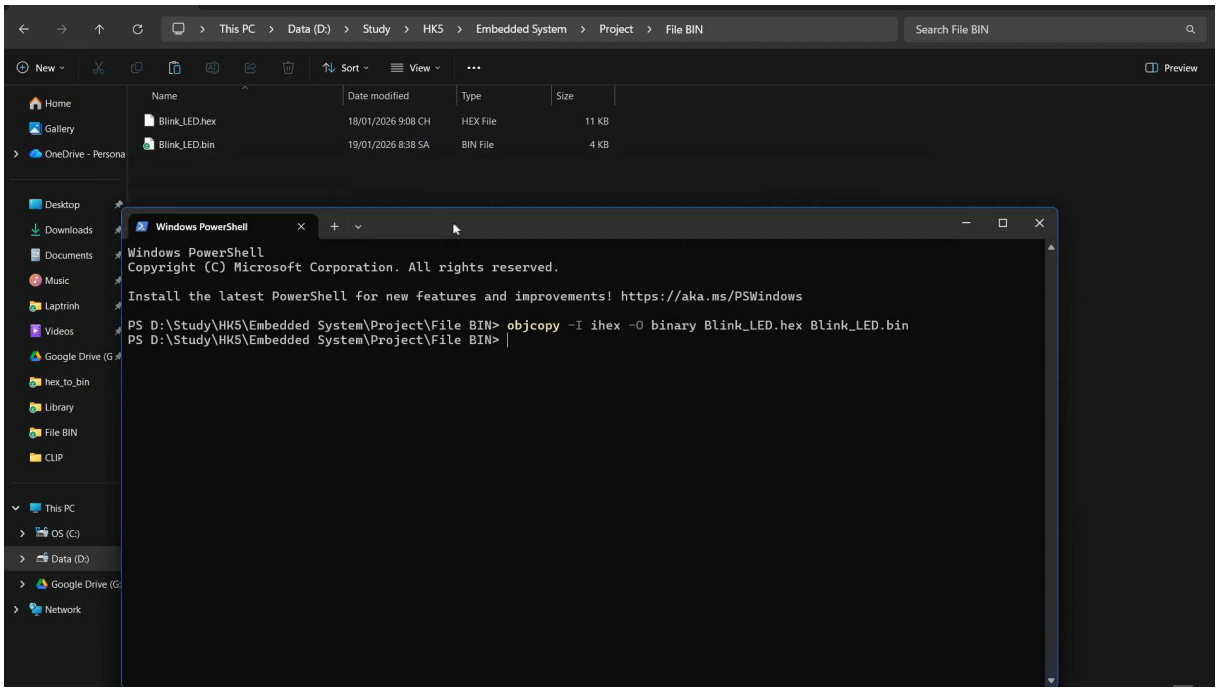
CPU chỉ thấy:

00100101100111011001100110101011001001001101010100010001...

Đây là chuỗi byte liên tục, không có dấu kết thúc, kí tự hay ranh giới. Giả sử file được nạp từ địa chỉ 0x0800 C800, các byte được ghi như sau:

Địa chỉ	**Byte**
0x0800C800	00100101
0x0800C801	10011101
0x0800C802	10011001
0x0800C803	10101011
0x0800C804	00100100
0x0800C805	11010101
0x0800C806	00010001
0x0800C807	...

Để thay đổi file dữ liệu từ dạng .hex sang .bin, chúng ta thực hiện câu lệnh như sau:



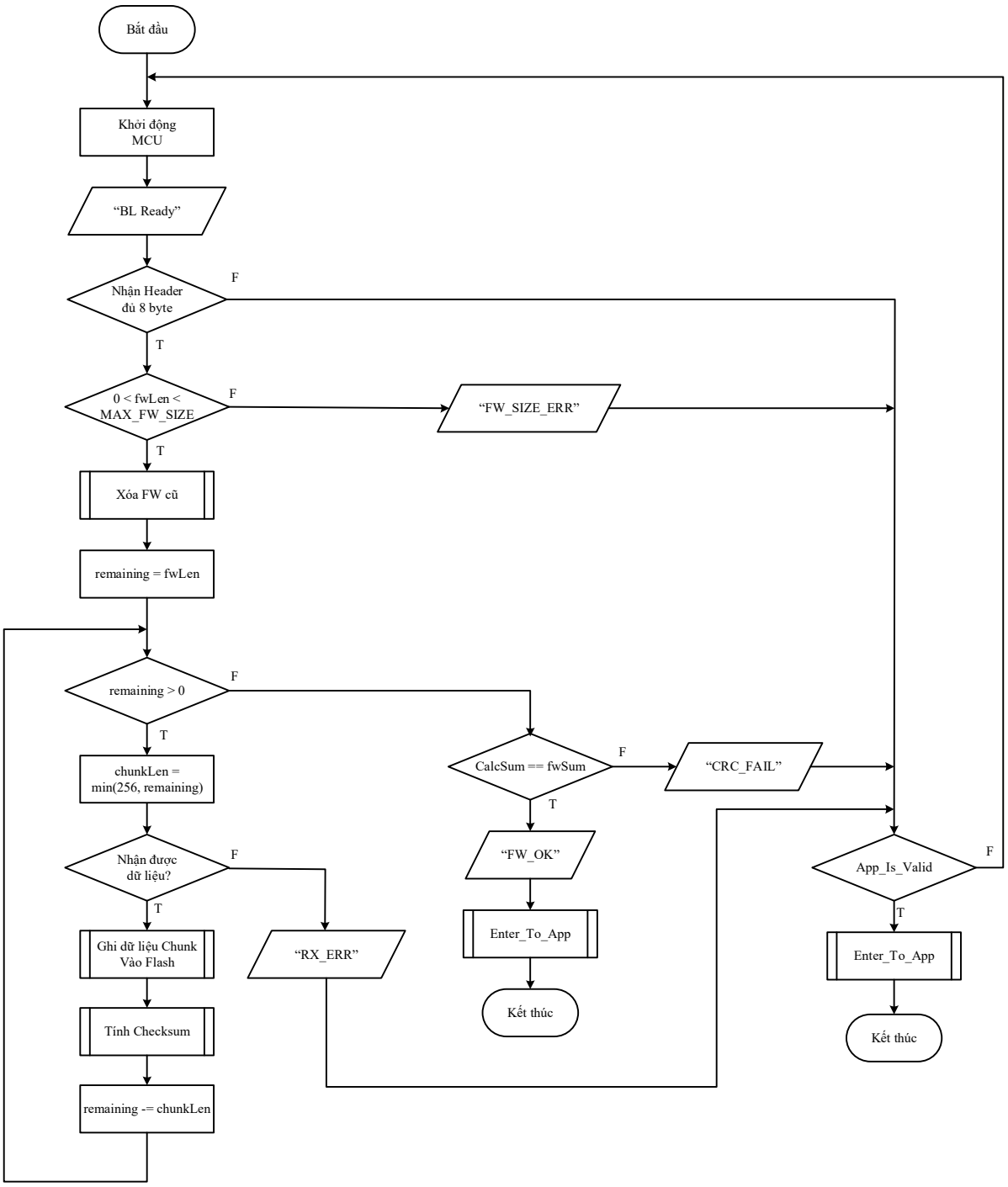
Hình 1-7 Câu lệnh chuyển đổi file .hex thành .bin

Lệnh: `objcopy -I ihex -O binary Blink_LED.hex Blink_LED.bin` được sử dụng để trích xuất dữ liệu nhị phân từ file HEX và tạo ra file BIN tương ứng.

→ Kết quả cho thấy file Blink_LED.bin đã được tạo thành công với dung lượng nhỏ hơn file HEX, phù hợp để sử dụng trong quá trình nạp firmware thông qua bootloader.

CHƯƠNG 2. THIẾT KẾ CHƯƠNG TRÌNH BOOTLOADER

2.1. Thuật toán chương trình Bootloader



Hình 2-1 Sơ đồ thuật toán Bootloader

- Lưu đồ chương trình Bootloader trên vi điều khiển STM32F103C8T6 thực hiện theo các bước như sau:

+ Bước 1: Khởi động & gửi tín hiệu BL Ready. Khi MCU được khởi động → bootloader khởi tạo các ngoại vi cần thiết như Clock, GPIO (LED A0 sáng), UART →

gửi một chuỗi ký tự “BL_READY” qua UART nhằm báo với lên máy tính rằng MCU sẵn sàng kết nối.

+ Bước 2: Nhận và kiểm tra Header. MCU chờ nhận header của firmware bao gồm kích thước firmware (fwSize) và giá trị checksum (fwSum).

• Nếu không được header trong 5s, bootloader nhảy đến của chương trình ứng dụng cũ.

• Nếu nhận được header, bootloader kiểm tra kích thước firmware. Nếu kích thước vượt quá dung lượng cho phép, bootloader gửi thông báo lỗi “FW_SIZE_ERR” và chuyển sang thực thi ứng dụng cũ.

• Nếu kích thước firmware hợp lệ, bootloader tiếp tục quá trình cập nhật firmware.

+ Bước 3: Xóa firmware cũ. Dựa trên fwSize nhận được và kích thước mỗi page (1KB), bootloader tính số lượng page cần xóa bằng cách làm tròn lên (ví dụ: 52.3 page cần xóa thì sẽ xóa 53 page). Sau đó, chương trình thực hiện mở khóa bộ nhớ flash → xóa dữ liệu theo từng page → khóa lại bộ nhớ.

+ Bước 4: Ghi dữ liệu firmware & tính toán checksum. Bootloader sẽ nhận dữ liệu firmware theo từng chunk. Mỗi chunk có độ dài bằng min (256,remaining).

• Nếu không nhận được chunk dữ liệu trong 5s, bootloader gửi tín hiệu “RX_ERR” và nhảy đến của chương trình ứng dụng cũ.

• Nếu nhận dữ liệu thành công, bootloader mở khóa bộ nhớ flash → ghi dữ liệu theo Half-Word (16 bit) → Khóa bộ nhớ flash.

• Thực hiện tính số byte cần nhận còn lại (remaining) và tổng checksum (calcSum).

• Tiếp tục lặp lại quá trình nhận cho đến hết dữ liệu firmware.

+ Bước 5: Kiểm tra Checksum. So sánh giá trị calcSum tính được và fwSum của header.

• Nếu khác nhau, bootloader gửi tín hiệu “CRC_FAIL” và nhảy đến chương trình cũ.

• Nếu giống nhau, bootloader gửi tín hiệu “FW_OK” và nhảy đến chương trình mới.

+ Bước 6: Kiểm tra Application cũ có hợp lệ: Nếu bootloader cần nhảy sang ứng dụng cũ, chương trình sẽ đọc giá trị con trỏ Stack Pointer và Reset Handle.

• Nếu một trong hai biến có giá trị là 0xFFFFFFFF, chương trình cũ không hợp lệ → chạy lại bootloader.

• Nếu không, chạy hàm nhảy sang Application.

+ Bước 7: Nhảy sang Application. Bootloader nhảy sang chương trình theo thứ tự sau:

• Bootloader tắt các khởi tạo các ngoại vi, cấu hình xung nhịp, cấu hình ngắt đã thiết lập.

• Thiết lập con trỏ Stack Pointer (MSP) đọc giá trị đầu tiên trong bảng vector của Application.

• Thiết lập địa chỉ hàm Reset Handler bằng địa chỉ Application + 4 bytes và nhảy vào App.

2.2. Giải thích chương trình con

2.2.1. Hàm ghi Flash

```
static void Flash_Write(uint32_t addr, uint8_t *mData, uint16_t bLen) {
uint8_t oFlag = 0; // co bao byte le
if (bLen % 2 == 1) {
     oFlag = 1;
     bLen--; // Giam ve so chan
}
for (uint32_t i = 0; i < bLen; i += 2) {
// Ghi data theo 1/2 word (16-bit) vidu 1234: 34 -> 12
uint16_t data = mData[i] | ((uint16_t)mData[i+1] << 8);
HAL_FLASH_Program(FLASH_TYPEPROGRAM_HALFWORD, addr + i, data);
}
if (oFlag == 1) {   // Ghi byte le cuoi cung (them 0xFF)
     uint16_t data = mData[bLen] | 0xFF00;
     HAL_FLASH_Program(FLASH_TYPEPROGRAM_HALFWORD, addr + bLen, data);
}
}
```

Bộ nhớ Flash chỉ hỗ trợ ghi theo đơn vị half-word (16 bit) nên hàm phải xử lý dữ liệu theo từng cặp 2 byte, không thể ghi từng byte riêng lẻ như RAM.

Nếu độ dài dữ liệu là số lẻ → đưa `bLen` về số chẵn và đặt cờ `oFlag` để xử lý sau vòng lặp chính.

Trong vòng lặp, hàm ghép hai byte liên tiếp trong `mData` thành một biến 16 bit theo chuẩn Little Endian → ghi xuống flash bằng hàm `HAL_FLASH_Program`.

Nếu có byte lẻ cuối cùng, hàm sẽ ghép byte này với giá trị đệm `0xFF` (trạng thái mặc định của Flash sau khi xóa) để tạo thành một half-word hợp lệ trước khi ghi xuống bộ nhớ.

Toàn bộ quá trình ghi yêu cầu vùng Flash đã được xóa trước đó và phải thực hiện mở khóa Flash trước khi ghi, sau khi hoàn tất thì khóa lại để đảm bảo an toàn bộ nhớ.

2.2.2. Hàm xóa Flash

```
static void Flash_Erase(uint32_t addr) {
	FLASH_EraseInitTypeDef pErase;
	pErase.NbPages = 1;
	pErase.PageAddress = addr;
	pErase.TypeErase = FLASH_TYPEERASE_PAGES;
	uint32_t PError;
	HAL_FLASHEx_Erase(&pErase, &PError);
}
```

Flash chỉ ghi được bit từ 1 → 0, vì vậy muốn ghi dữ liệu mới bắt buộc phải xóa trước để đưa toàn bộ bit về trạng thái 0xFF.

Cấu trúc `FLASH_EraseInitTypeDef` dùng để cấu hình thao tác xóa gồm kiểu xóa theo page, địa chỉ page bắt đầu và số page cần xóa.

Biến `PError` dùng để lưu thông tin page bị lỗi nếu quá trình xóa không thành công.

Hàm `HAL_FLASHEx_Erase()` thực hiện lệnh xóa Flash ở mức phần cứng, yêu cầu Flash phải được unlock trước và lock lại sau khi hoàn tất.

2.2.3. Hàm xóa Firmware

```
static void Erase_Firmware(uint32_t fwLen) {
      HAL_FLASH_Unlock();
      uint32_t erase_addr = ADDR_APP_PROGRAM;
      // Lam tron len
      uint32_t nbPages = (fwLen + PAGE_SIZE - 1) / PAGE_SIZE;
      for (uint32_t page = 0; page < nbPages; page++) {
        Flash_Erase(erase_addr);
        erase_addr += PAGE_SIZE;
      }
HAL_FLASH_Lock();
}
```

Hàm dùng để xóa toàn bộ vùng Flash dự kiến chứa firmware mới, bắt đầu từ địa chỉ ADDR_APP_PROGRAM.

Flash được mở khóa (HAL_FLASH_Unlock) trước khi thao tác và khóa lại (HAL_FLASH_Lock) sau khi hoàn tất để đảm bảo an toàn bộ nhớ.

Số page cần xóa được tính bằng công thức làm tròn lên nbPages để đảm bảo xóa đủ cả phần firmware không tròn page.

Vòng lặp thực hiện xóa từng page một bằng cách gọi lại hàm Flash_Erase() và tăng địa chỉ theo từng kích thước page.

Sau khi hoàn tất, toàn bộ vùng nhớ chứa firmware cũ đã được đưa về trạng thái trống (0xFF) sẵn sàng ghi firmware mới.

2.2.4. Hàm nhận và ghi Firmware

```
static uint32_t Receive_And_Write_Firmware(uint32_t fwLen) {
    uint8_t rx_buf[256]; // Buffer dem 256 bytes
    uint32_t addr = ADDR_APP_PROGRAM;
    uint32_t remaining = fwLen;
    uint32_t calc_sum = 0;
    HAL_FLASH_Unlock();
    while (remaining > 0) {
        // Tinh do dai Chunk
        uint16_t ChunkLen = (remaining > sizeof(rx_buf)) ? sizeof(rx_buf) :
remaining;
```

```
         if (HAL_UART_Receive(&huart1, rx_buf, ChunkLen, UART_TIMEOUT_MS) !=
HAL_OK) {
            // Nhan that bai
            HAL_FLASH_Lock(); // Lock lai truoc khi thoat
            const char *err_msg = "RX_ERR\r\n";
            HAL_UART_Transmit(&huart1, (uint8_t*)err_msg, strlen(err_msg),
HAL_MAX_DELAY);
        if (Is_App_Valid()) Enter_To_App(); // chay lai app cu
        }
        else {
            // Nhan thanh cong -> Ghi Flash
            Flash_Write(addr, rx_buf, ChunkLen);

            calc_sum += Calc_Checksum(rx_buf, ChunkLen);
            addr += ChunkLen;
            remaining -= ChunkLen;
        }
    }
    HAL_FLASH_Lock();
    return calc_sum;
}
```

Hàm có nhiệm vụ nhận dữ liệu firmware từ UART theo từng chunk → ghi tuần tự xuống Flash → tính tổng checksum của toàn bộ dữ liệu đã nhận.

Bộ nhớ Flash được mở khóa trước khi ghi và chỉ khóa lại sau khi quá trình ghi hoàn tất hoặc khi xảy ra lỗi.

Trong mỗi vòng lặp, kích thước dữ liệu nhận `ChunkLen = min(sizeof(buf), remaining)`.

Nếu quá trình nhận thất bại → hệ thống dừng ghi Flash → gửi thông báo lỗi "RX_ERR" → chạy lại application cũ (nếu hợp lệ).

Nếu nhận thành công → ghi xuống Flash bằng hàm `Flash_Write()` → cập nhật địa chỉ ghi tiếp theo, số byte còn lại, và cộng dồn checksum.

Giá trị trả về của hàm là checksum tính được từ firmware thực tế, dùng để so sánh với checksum gửi từ máy tính.

2.2.5. Hàm Is_App_Valid()

```
static int Is_App_Valid(void) {
	uint32_t sp = (__IO uint32_t)ADDR_APP_PROGRAM;
	uint32_t reset = (__IO uint32_t)(ADDR_APP_PROGRAM + 4);
	if (sp == 0xFFFFFFFFU || reset == 0xFFFFFFFFU){
		return 0;// ko hop le;
	}
return 1;
}
```

Hàm dùng để kiểm tra xem firmware Application trong Flash có hợp lệ hay không trước khi cho phép Bootloader nhảy sang chạy chương trình đó.

Chương trình đọc 2 giá trị đầu tiên tại địa chỉ bắt đầu của vùng Application: giá trị thứ nhất là MSP (Main Stack Pointer) và giá trị thứ hai là địa chỉ hàm Reset Handler.

Hai giá trị này được truy cập thông qua con trỏ trỏ trực tiếp vào bộ nhớ Flash ((__IO uint32_t*)ADDR_APP_PROGRAM), vì vector table của một chương trình ARM luôn nằm ở đầu vùng nhớ chương trình.

Nếu một trong hai giá trị đọc được bằng 0xFFFFFFFF (trạng thái Flash chưa ghi), điều đó cho thấy vùng Application chưa có, nên hàm sẽ trả về 0.

Ngược lại, nếu cả hai giá trị đều hợp lệ, hàm trả về 1 để báo rằng Application có thể được thực thi an toàn.

2.2.6. Hàm Enter_To_App()

```
static void Enter_To_App(void) {
	// Tat cac ngat dang hoat dong
	HAL_RCC_DeInit(); HAL_DeInit();
	//Xoa cac co loi
	SCB->SHCSR &=~(	SCB_SHCSR_USGFAULTENA_Msk|
			SCB_SHCSR_BUSFAULTENA_Msk|
			SCB_SHCSR_MEMFAULTENA_Msk);
	// Doi Vector Table ve vung nho App
	SCB->VTOR = ADDR_APP_PROGRAM;
	// Dat gia tri Main Stack Pointer (MSP)
	__set_MSP(*(__IOuint32_t*)ADDR_APP_PROGRAM);
	// Lay dia chi Reset Handler cua App va nhay vao
	pFunction app_entry = (pFunction)(*(__IO uint32_t*)
```

```
    (ADDR_APP_PROGRAM + 4));
    app_entry();
}
```

Hàm xóa mặc định de-init Clock và các ngoại vi HAL → tránh ảnh hưởng đến Application.

Khi nhảy sang Application mới, cờ lỗi được vô hiệu hóa → tránh nhảy sang ngắt không mong muốn.

Cập nhật giá trị bắt đầu Vector Table về địa chỉ vùng nhớ Application → MCU dung Vector Table của chương trình mới.

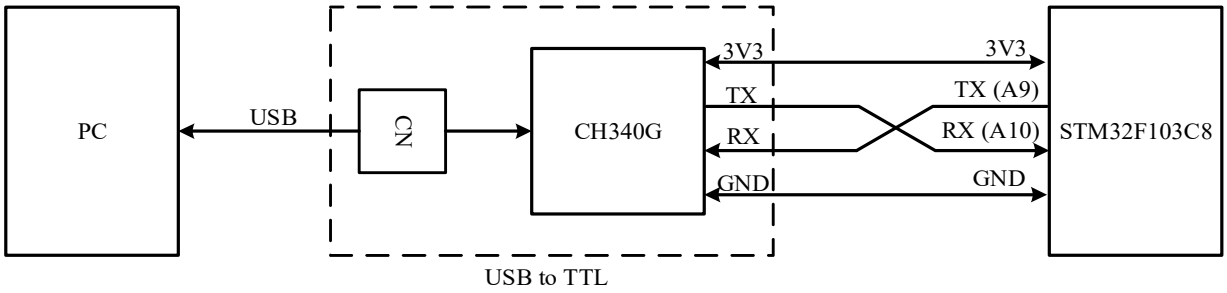
Giá trị Main Stack Pointer (MSP) được nạp lại từ 4 byte đầu tiên của vector table.

Cuối cùng, Bootloader đọc địa chỉ Reset Handler của Application (4 byte tiếp theo của vector table) → ép kiểu thành con trỏ hàm → gọi hàm để reset MCU → chạy chương trình mới.

CHƯƠNG 3. THIẾT KẾ CÔNG CỤ PYTHON CẬP NHẬT FIRMWARE QUA UART

Để có thể cập nhật firmware mới dễ dàng, không cần thay đổi chế độ BOOT trên STM32, chúng ta sẽ xây dựng công cụ Python giúp cập nhật firmware qua chuẩn giao tiếp UART.

3.1. Sơ đồ khối và cấu trúc gói tin



Hình 3-1 Sơ đồ khối kết nối tool nạp firmware qua UART

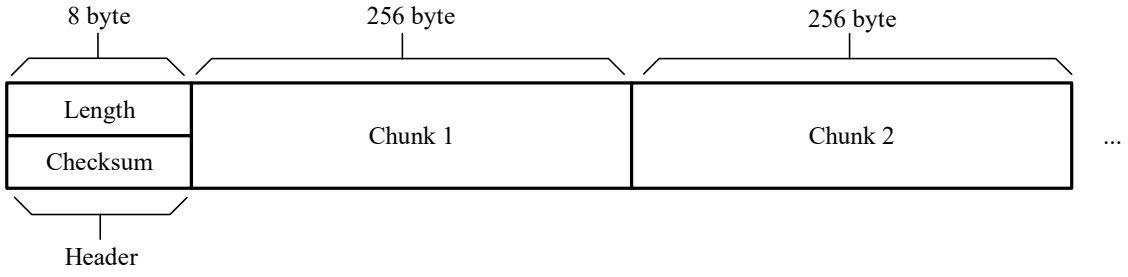
Sơ đồ mô tả cách kết nối phần cứng phục vụ quá trình nạp firmware cho vi điều khiển STM32F103C8 bằng công cụ Python qua UART.

+ PC đóng vai trò là thiết bị thực thi lệnh Python để giao tiếp, đóng gói và truyền firmware đến STM32 qua cổng USB.

+ Module USB to TTL có chức năng chuyển đổi tín hiệu của cổng USB sang giao thức UART, cấp nguồn 3V3 và GND cho vi điều khiển. Để thực hiện chức năng trên, module sử dụng chip chuyển đổi CH340G. Do đó, PC cần phải có driver chuyển đổi USB sang TTL của dòng chip CH340 để giao tiếp được với module.

+ STM32F103C8 là thiết bị đích nhận firmware được truyền từ PC. Vi điều khiển sử dụng hai chân A9 (TX) và A10 (RX) để giao tiếp qua giao thức UART.

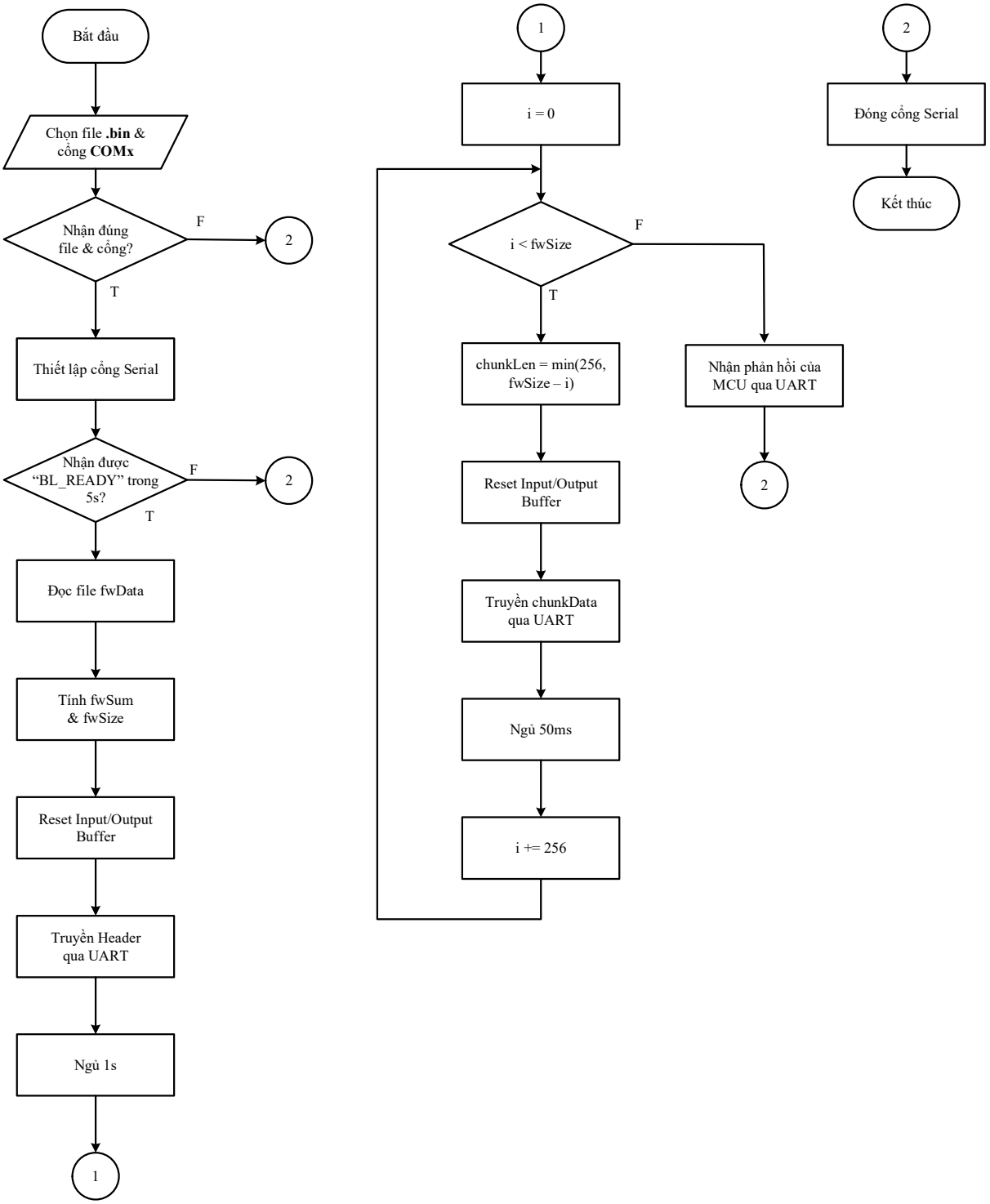
Quá trình thực hiện truyền firmware lên vi điều khiển được mô tả dựa trên sơ đồ khối tổng quan như sau:



Hình 3-2 Cấu trúc gói tin truyền Firmware

Gói tin bao gồm phần header 8 byte được đóng gói từ hai phần có dung lượng bằng 4 byte: độ dài file Firmware và Checksum. Theo sau đó, Firmware được chia thành các Chunk có độ dài 256 byte và được truyền theo kiểu Little-Endian.

3.2. Lưu đồ thuật toán của tool nạp Python



Hình 3-3 Lưu đồ thuật toán của công cụ nạp firmware lên MCU qua UART

- Lưu đồ thuật toán tool Python nạp bootloader được thực hiện theo các bước như sau:

+ Bước 1: Thiết lập kết nối và chờ tín hiệu “BL_Ready”.

- Tool Python nhận file firmware và cổng COM giao tiếp với MCU → Nếu không nhận được, chương trình đóng kết nối. Ngược lại, PC sẽ thiếp lập cổng UART.
- PC sẽ chờ tín hiệu BL_READY từ MCU trong 5s. → Nếu không nhận được, chương trình đóng cổng Serial và ngắt kết nối. Ngược lại, PC sẽ tiếp hành bước kế tiếp.

+ Bước 2: Đọc dữ liệu firmware theo kiểu nhị phân và gửi header.

- Chương trình sẽ đọc cổng file theo dạng nhị phân, tín toán độ dài file (fwLen) và checksum (fwSum).
- Đóng gói fwLen và fwSum thành file header có độ dài 8 byte.
- Xóa buffer nhận/truyền trên cổng Serial.
- Truyền Header và chờ 1s cho MCU nhận được header.

+ Bước 3: Gửi firmware theo từng chunk.

- Thực hiện vòng lặp gửi từng gói tin sang MCU với độ dài tối đa 256 byte.
- Trong mỗi lần gửi đều xóa buffer truyền/nhận của PC và ngủ 50ms cho MCU thực hiện được ghi và xóa.

+ Bước 4: Chờ phản hồi từ MCU và kết thúc.

- Chờ nhận gói tin phản hồi từ MCU và hiển thị cho người dùng kết quả truyền firmware thành công hoặc thất bại.
- Đóng cổng kết nối Serial và kết thúc chương trình

3.3. Giải thích hàm trong chương trình

3.3.1. Bắt tay và thiết lặp kết nối với MCU.

```
ser = serial.Serial(serial_port, BAUDRATE, bytesize=8, parity='N', stopbits=1, timeout=5)
    time.sleep(3)
    print(f"[PC] Waiting for BL_READY on {serial_port}...")

    # Chờ MCU báo BL_READY
    for _ in range(5):
        bl_msg = ser.readline().decode(errors="ignore").strip()
        print(f"[MCU] {bl_msg}")
```

```
        if "BL_READY" in bl_msg:
            break
    else:
        messagebox.showerror("Lỗi", "Bootloader không phản hồi!")
        ser.close()
        return
```

Tool Python mở cổng Serial với giá trị BAUDRATE = 115200, 8 bit data, 1 stop bit, 0 bit parity, thời gian time out 5s. → Tạm dừng 3s để chờ kết nối với MCU.

Trong 5s tiếp theo, chờ nhận tín hiệu "BL_READY" được giải mã bỏ qua các thành phần lỗi và các dấu xuống dòng (\n).

Nếu nhận được, tool thoát khỏi vòng lặp và chạy bước tiếp theo. Ngược lại, chương trình báo lỗi và đóng cổng kết nối.

3.3.2. Đọc dữ liệu firmware và gửi header

```
    with open(fw_file, "rb") as f:
        fw_data = f.read()

    fw_size = len(fw_data)
    fw_sum = calc_checksum(fw_data)

    print(f"[PC] Firmware size: {fw_size} bytes")
    print(f"[PC] Checksum    : 0x{fw_sum:08X}")

    header = struct.pack("<II", fw_size, fw_sum)
    ser.write(header)
    time.sleep(1)
```

Dữ liệu được đọc theo kiểu nhị phân ("rb") và tính toán độ dài firmware với checksum cơ bản (tổng các bit trong firmware).

Header được đóng gói bằng hàm `struct.pack`, với dấu `<` biểu thị kiểu đóng gói Little Endian, `II` biểu diễn ghép 2 giá trị kiểu Unsigned integer (mỗi giá trị 4 byte).

Sau đó được gửi qua Serial và ngủ 1s để MCU có thể đọc và xử lý.

3.3.3. Tuyền Header theo từng chunk

```
ChunkLen = 256
for i in range(0, fw_size, ChunkLen):
    Chunk = fw_data[i:i+ChunkLen]
    ser.reset_input_buffer()
    ser.reset_output_buffer()
    ser.write(Chunk)
    print(f"[PC] Sent Chunk {i//ChunkLen}, size {len(Chunk)} bytes")
    time.sleep(0.05)
```

Để gửi dữ liệu theo từng chunk, chúng ta cho vòng lặp for với mỗi lần biến đếm `i` tăng thêm `256 bytes`.

Từng chunk được giới hạn bởi giá trị biến đếm `i` và giá trị `i + 256`. Tại thời điểm giá trị `i + ChunkLen` vượt quá độ dài của firmware, hàm của python tự động gán giá trị trên bằng `fw_size` để tránh bị lỗi.

Mỗi lần gửi cần xóa buffer nhận/ghi trên cổng Serial của máy tính và cho chương trình delay 50ms để MCU xử lý kịp.

CHƯƠNG 4. KẾT QUẢ VÀ ĐÁNH GIÁ

4.1. Quy trình cập nhật firmware qua UART sử dụng bootloader

Sau khi khởi tạo thành công chương trình bootloader, chúng ta tiến hành nạp chương trình thông qua phần mềm Flash Loader Demonstrator.

STM32 được kết nối với máy tính qua các đường TX, RX và GND chung, đồng thời được đưa vào chế độ System Memory Boot bằng cách thiết lập BOOT0 = 1 và BOOT1 = 0, sau đó reset vi điều khiển.

Tiếp đến, thiết lập phần mềm Flash Loader Demonstrator với thông số kết nối như sau:



Hình 4-1 Các thông số trên phần mềm Flash Loader Demonstrator

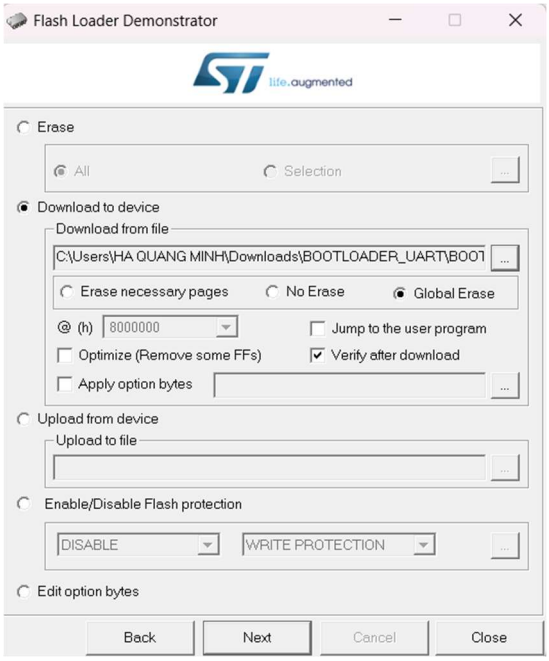
Port Name: Chọn cổng COM mà STM32 đang kết nối

Baud Rate: 115200: Tốc độ truyền phù hợp với cấu hình USART1 đã thiết lập trong STM32CubeMX.

Data Bits: 8, Parity: Even. Đây là yêu cầu bắt buộc của bootloader ROM STM32 khi giao tiếp UART .

Echo: Disabled, Timeout(s): 10. Thời gian chờ đủ dài để đảm bảo quá trình handshake và truyền dữ liệu không bị gián đoạn.

Nếu cấu hình phần cứng và thông số UART chính xác, Flash Loader Demonstrator sẽ thực hiện bắt tay, tự động nhận dạng dòng vi điều khiển, dung lượng Flash và các vùng nhớ khả dụng.



Hình 4-2 Nạp file .hex vào MCU

Chọn chức năng Download to device Tải file bootloader .hex đã được biên dịch từ KeilC → Kiểm tra địa chỉ nạp: Bootloader được đặt ở đầu Flash (0x08000000) → Chọn tùy chọn: Global erase để đảm bảo Flash trống trước khi nạp → Sau khi kiểm tra lại các thông số, nhấn Next để bắt đầu quá trình ghi Flash.



Hình 4-3 Nạp Booloader thành công

Phần mềm sẽ ghi dữ liệu bootloader vào Flash, tự động kiểm tra lại dữ liệu sau khi ghi. Nếu quá trình hoàn tất thành công, Flash Loader Demonstrator sẽ hiển thị thông báo Download successful.

Sau khi bootloader đã được nạp, vi điều khiển được cấu hình lại để khởi động từ Flash bằng cách đưa BOOT0 về mức 0 và reset hệ thống. Từ thời điểm này, STM32 sẽ thực thi bootloader do người dùng phát triển.

4.2. Cập nhật firmware bằng chương trình Python

Để thực hiện quá trình này, một chương trình Python được xây dựng nhằm tự động hóa việc gửi firmware từ máy tính xuống vi điều khiển. Trước hết, chúng ta cần xây dựng một Application đơn giản trước.

```
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART1_UART_Init();
  /* USER CODE BEGIN 2 */
  SCB->VTOR = 0x0800C800;
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_0);
    HAL_Delay(500);
    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
```

Hình 4-4 Chương trình BLINK_LED cơ bản

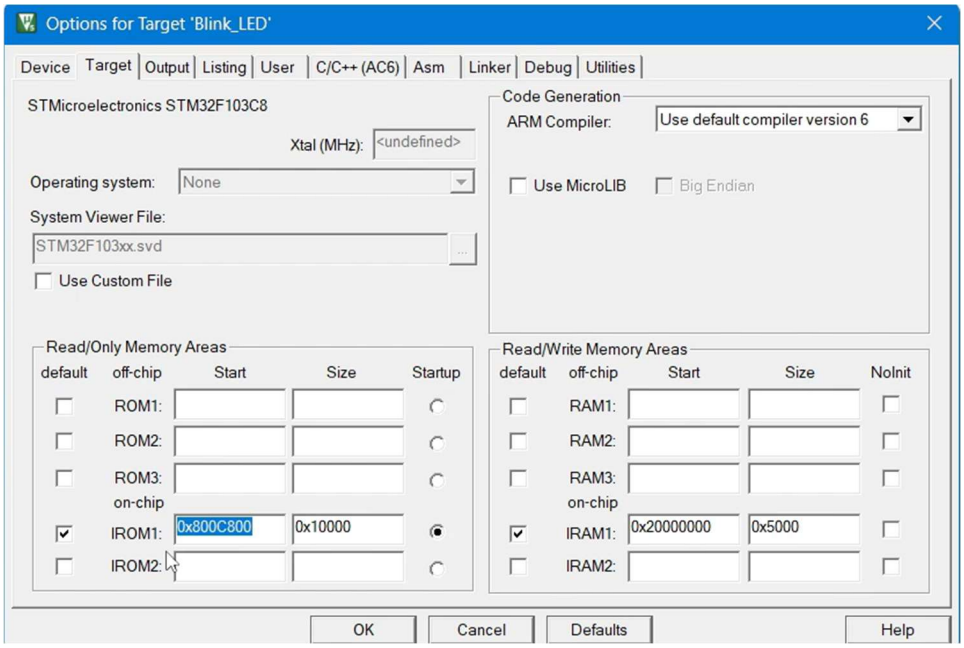
Trong application này, địa chỉ Vector Table có giá trị là 0x0800C800. Lựa chọn địa chỉ này nhằm:

+ Chừa cho bootloader một vùng Flash lớn từ 0x8000000 → 0x800C800 (dung lượng bootloader tối đa 50kB). Trong BootLoader hình 4-3 chỉ dung lượng 6.58 kB nên đủ chứa trong vùng trên.

+ Địa chỉ phải nằm trên địa chỉ bắt đầu của một page → dễ dàng ghi xóa dữ liệu hơn.

+ Địa chỉ dành cho application là: 0x0800C800 → 0x0800FFFF dung lượng (tối đa 14 kB). Tuy nhiên, chương trình chỉ nhấp nháy led cơ bản chỉ tốn 4kB dữ liệu nên vẫn chứa đủ.

Trong các trường hợp khác, nếu dung lượng Application lớn, ta có thể thay đổi địa chỉ bắt đầu trong khoảng từ: 0x08001C00 (địa chỉ bắt đầu page 7) → 0x0800FC00 (địa chỉ bắt đầu page 63).



Hình 4-5 Bảng Options for Targets của Application

- Khi chỉnh trong Options for Target → Target: IROM1 Start Address = 0x0800C800, ta đang cài đặt để toàn bộ các section của chương trình application phải được đặt bắt đầu từ địa chỉ 0x0800C800.

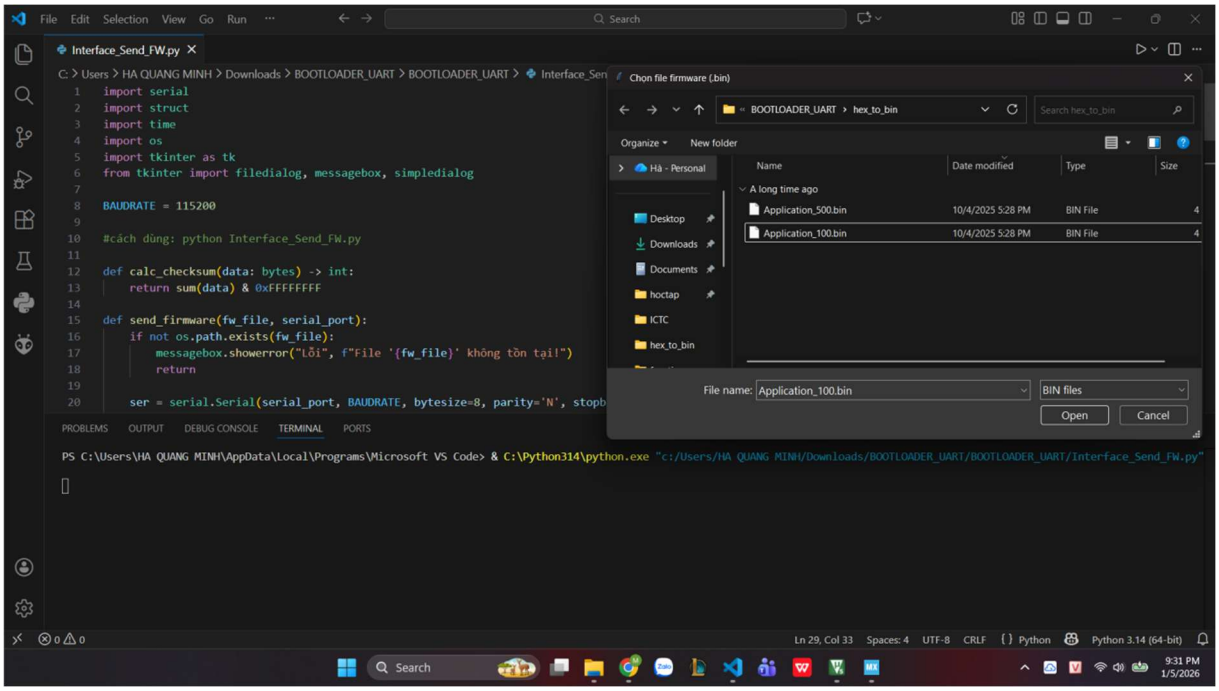
- Khi cài đặt xong:

+ Toàn bộ mã application được liên kết bắt đầu từ 0x0800C800.

+ Vector table của application cũng nằm tại địa chỉ 0x0800C800.

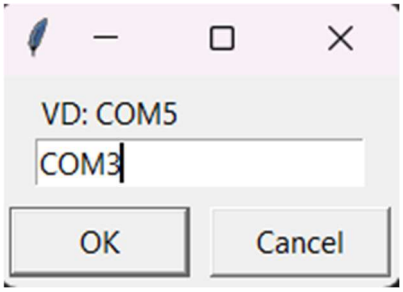
+ Các địa chỉ hàm, biến và vector ngắt được tính toán tương ứng với địa chỉ mới.

- Dù File .bin chỉ chứa dữ liệu thô và không có thông tin địa chỉ, nhưng vị trí nạp chương trình cần được thay đổi trực tiếp trong Application dựa trên giá trị IROM1 và địa chỉ thanh ghi Vector Table trong KeilC. Khi đó, các giá trị trên của Application mới khớp với giá trị đã được cài đặt trong BootLoader → chương trình mới hoạt động được.



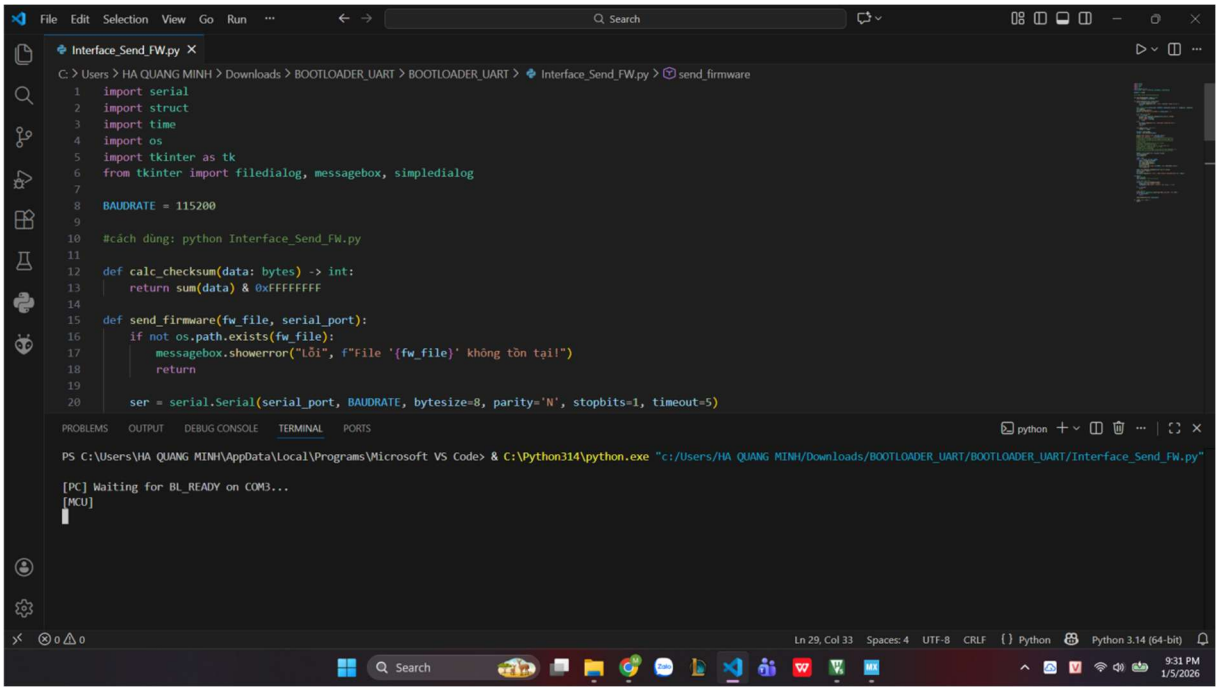
Hình 4-6 Hộp thoại chọn file .bin

Sau khi tạo ra file .bin của firmware mới, chúng ta tiến hành chạy tool Python. Tại đây, chương trình hiển thị hộp thoại chọn file firmware (.bin) và chúng ta chọn file firmware .bin của mình mong muốn.



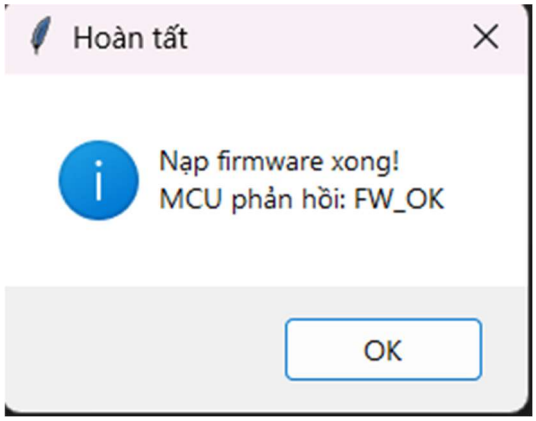
Hình 4-7 Hộp thoại nhập cổng COM tương ứng với STM32

Chương trình yêu cầu kết nối với cổng giao tiếp USB TTL và có các tham số như sau: Baud rate: 115200 bps; Data bits: 8; Parity: None; Stop bits: 1; Timeout: 5 giây.



Hình 4-8 Chờ tín hiệu BL_READY từ MCU

Sau khi kết nối, chương trình chờ tín hiệu BL_READY từ STM32, xác nhận rằng bootloader đã khởi động thành công và sẵn sàng nhận firmware. Nếu không nhận được phản hồi trong thời gian cho phép, quá trình sẽ bị dừng và báo lỗi.



Hình 4-9 Kết quả nạp firmware

FW_OK: Firmware hợp lệ, quá trình nạp thành công, MCU nhảy sang application mới; CRC_FAIL: Dữ liệu bị lỗi trong quá trình truyền hoặc ghi Flash.

CHƯƠNG 5. KẾT LUẬN

Quy trình cập nhật bootloader và firmware qua UART đã xây dựng theo các bước rõ ràng, ổn định. Bootloader sau khi được nạp vào MCU có thể hoạt động độc lập, cho phép cập nhật firmware mà không cần sử dụng mạch nạp. Kết quả này xác nhận Bootloader đã vận hành thành công, tối ưu hóa việc quản lý và bảo trì thiết bị.

DANH MỤC TÀI LIỆU THAM KHẢO

[1] Hoang Manh Hung, Pham Quang Phuc, Duong Minh Tri, Phan Khac Hung, Nguyen Truong Minh Quan, "[STM32] [7S-N6] Bootloader," 16 10 2025. [Trực tuyến]. Available: https://youtube.com/playlist?list=PL9JrMSL4UPJ8wSOqp8Hn0DsSd2hvytJ_g&si=SK4tz1BAlpb-uLyP. [Đã truy cập 18 12 2025].